Predviđanje zagađenja vazduha (PM2.5 čestica) korištenjem mašinskog učenja

Sasa Lakic RA220-2023

26. februar 2026.

1 Implementacija programa

U ovom poglavlju opisan je proces učitavanja i inicijalne obrade podataka korišćenjem biblioteke `pandas` u jeziku Python.

1.1 Učitavanje i formatiranje vremenskih serija

Prvi korak u svakoj analizi vremenskih serija je pravilno rukovanje datumima. U nastavku je prikazana funkcija za učitavanje podataka:

```python
def load_air_quality_data(file_path):
df = pd.read_csv(file_path)

# Spajanje kolona u jedinstven datum
df['date'] = pd.to_datetime(df[['year', 'month', 'day', 'hour']])
df.set_index('date', inplace=True)

# Selektovanje relevantnih kolona
relevant_cols = ['PM2.5', 'TEMP', 'PRES', 'DEWP', 'RAIN', 'WSPM']
df = df[relevant_cols]

return df
```

Listing 1: Funkcija za učitavanje podataka

1.1.1 Obrazloženje postupka učitavanja

Korišćenje funkcije `pd.read_csv` omogućava efikasno prebacivanje podataka iz tekstualnog formata u `DataFrame` strukturu koja je optimizovana za rad sa velikim količinama podataka. Ključni koraci u ovoj fazi su:

- **Konverzija u `datetime`:** Funkcija `pd.to_datetime` spaja odvojene kolone (godina, mjesec, dan, sat) u jedinstven vremenski objekat. Ovo je neophodno jer algoritmi poput LSTM-a i Prophet-a moraju prepoznati hronološki redoslijed i cikličnost (npr. prelaz sa 23:00 na 00:00 narednog dana).
- **Postavljanje indeksa (`set_index`):** Pretvaranjem kolone `date` u indeks, podaci se transformišu iz obične tabele u **vremensku seriju (Time Series)**. Ovo drastično olakšava kasnije operacije poput interpolacije nedostajućih vrijednosti i crtanja grafika, jer `pandas` tada automatski prepoznaje vremensku distancu između uzoraka.

1.2 Pretprocesiranje i rješavanje problema nedostajućih vrijednosti

Sirovi podaci često sadrže nedostajuće vrijednosti (NaN) koje mogu uzrokovati greške pri treniranju modela. Umjesto prostog brisanja redova, što bi narušilo kontinuitet vremenske serije, primijenjena je dvo-fazna strategija imputacije unutar funkcije `preprocess_missing_values`.

```python
def preprocess_missing_values(df):
# 1. Linearna interpolacija za kratke prekide
df['PM2.5'] = df['PM2.5'].interpolate(method='linear', limit=2)

# Privremene kolone za sezonsku analizu
df['month'] = df.index.month
df['hour'] = df.index.hour

# 2. Imputacija na osnovu istorijskog prosjeka za dati sat i mjesec
df['PM2.5'] = df['PM2.5'].fillna(
df.groupby(['month', 'hour'])['PM2.5'].transform('mean')
)

```

```
    df.drop(['month', 'hour'], axis=1, inplace=True)
    return df
```

Listing 2: Metode za popunjavanje rupa u podacima

1.2.1 Obrazloženje strategije čišćenja

- **Linearna interpolacija (`limit=2`):** Ova metoda se koristi za popunjavanje kratkih prekida (1 do 2 uzastopna sata). Pretpostavka je da se koncentracija PM2.5 čestica ne mijenja trenutno, već postepeno, pa je srednja vrijednost između dva susjedna sata logična aproksimacija.
- **Sezonska imputacija (`groupby`):** Za duže periode nedostatka podataka, koristi se prosjek za taj specifični sat u tom mjesecu. Na primjer, ako nedostaje podatak za 14:00h u januaru, sistem izračunava prosjek svih dostupnih mjerenja u 14:00h tokom svih januara u setu podataka. Time se čuva dnevni i sezonski profil zagađenja koji je ključan za rad Prophet modela.
- **Očuvanje dimenzionalnosti:** Na kraju funkcije, ukupan broj uzoraka ostaje nepromijenjen (35,064), što potvrđuje da nismo gubili informacije brisanjem redova.

2 Eksplorativna analiza podataka (EDA)

Prije implementacije prediktivnih modela, izvršena je analiza korelacije i dekompozicija vremenske serije kako bi se identifikovali ključni faktori uticaja i sezonski obrasci.

2.1 Analiza korelacije meteoroloških faktora

Funkcija `run_eda_correlation` generiše matricu korelacije koja prikazuje linearnu povezanost između koncentracije PM2.5 čestica i atmosferskih prilika.

```
def run_eda_correlation(df):
plt.figure(figsize=(10,8))
corr_matrix = df.corr()
sb.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt=".2f")
plt.title("Korelacija PM2.5 estica sa meteorolo kim faktorima")
```

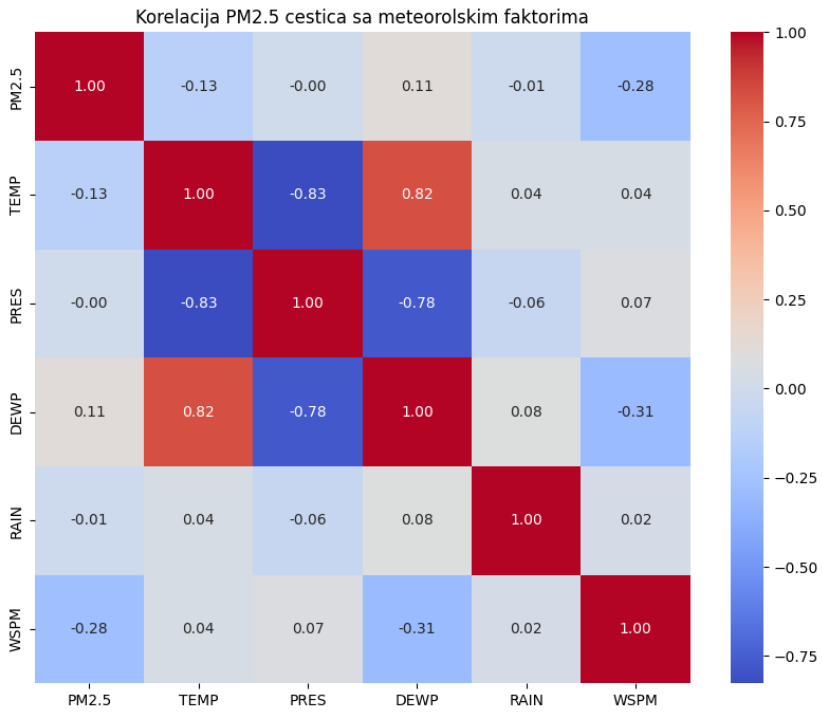
```
plt.show()
```

Listing 3: Vizuelizacija korelacije pomoću toplotne mape

2.1.1 Interpretacija rezultata korelacije

Na osnovu generisane toplotne mape (*heatmap*), uočene su ključne zavisnosti:

- **Negativna korelacija sa brzinom vjetra (WSPM):** Jači vjetar doprinosi disperziji čestica, čime se smanjuje njihova koncentracija u vazduhu.
- **Korelacija sa temperaturom (TEMP):** Niže temperature su često povezane sa povećanim zagađenjem uslijed fenomena temperaturne inverzije i povećane potrošnje energenata za grijanje.



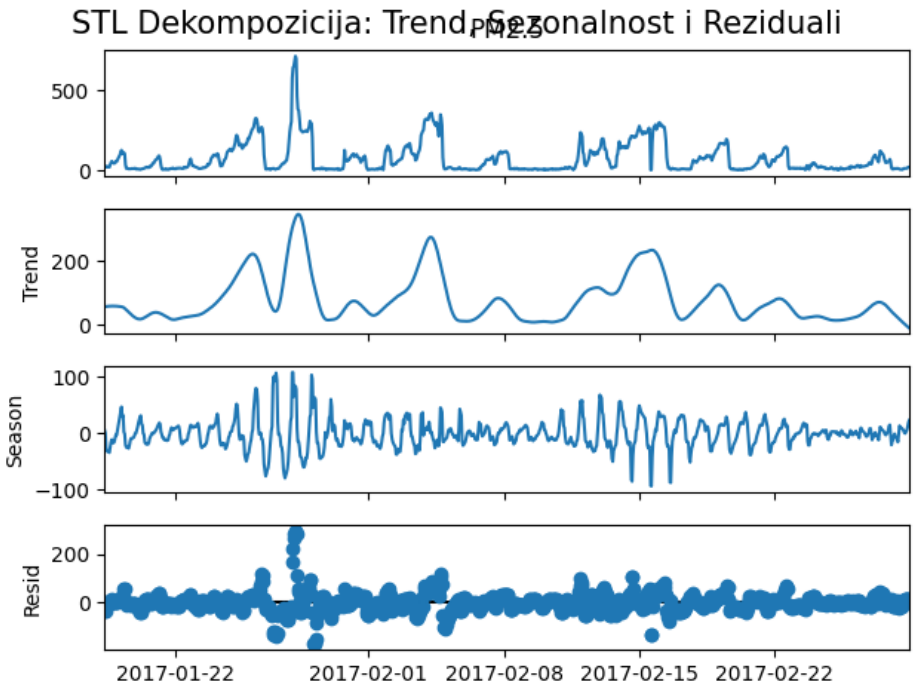
Slika 1: Matrica korelacije PM2.5 čestica i meteoroloških parametara

2.2 STL Dekompozicija

Za dublje razumijevanje strukture vremenske serije korišćena je **STL dekompozicija** (*Seasonal-Trend decomposition using LOESS*).

```python
def run_stl_decomposition(df):
subset = df['PM2.5'].tail(1000)
# period=24 predstavlja dnevni ciklus (24 sata)
res = STL(subset, period=24).fit()
fig = res.plot()
plt.suptitle('STL Dekompozicija: Trend, Sezonalnost i Reziduali')
plt.show()
```

Listing 4: Dekompozicija vremenske serije



Slika 2: STL dekompozicija vremenske serije PM2.5 (Trend, Sezonalnost i Reziduali)

2.2.1 Analiza komponenti dekompozicije

Na osnovu grafika prikazanog na Slici 2, možemo identifikovati tri ključna elementa serije:

- **Trend:** Prikazuje dugoročno kretanje nivoa zagađenja, očišćeno od dnevnih fluktuacija.
- **Seasonal (Sezonalnost):** Jasno su vidljivi pravilni oscilatorni kretaji sa periodom od 24 sata, što potvrđuje snažan uticaj dnevnih ljudskih aktivnosti i meteoroloških ciklusa na koncentraciju čestica.
- **Resid (Reziduali):** Predstavljaju ostatak varijanse koji se ne može objasniti trendom niti sezonalnošću (npr. nagle promjene vremena ili incidentna zagađenja).

3 ARIMA Model (Benchmark)

Kao bazni (*benchmark*) model za predviđanje koncentracije PM2.5 čestica odabran je **ARIMA** model. Njegova uloga je da pruži osnovnu metriku preciznosti zasnovanu na čisto statističkim metodama analize vremenskih serija.

```
def run_arima_model(df):
# Podjela na trening i test (zadnjih 6 mjeseci za test)
test_size = 4320
train = df['PM2.5'].iloc[:-test_size]
test = df['PM2.5'].iloc[-test_size:]

# Konfiguracija modela: (p, d, q) = (1, 1, 1)
model = ARIMA(train, order=(1, 1, 1))

# Optimizacija brzine izvrsavanja pomocu 'innovations_mle'
model_fit = model.fit(method='innovations_mle')

# Predvidjanje naredna 24 sata
forecast = model_fit.forecast(steps=24)
```

Listing 5: Implementacija ARIMA modela

3.1 Parametri i optimizacija

U modelu je korišćen **order=(1, 1, 1)**, što podrazumijeva:

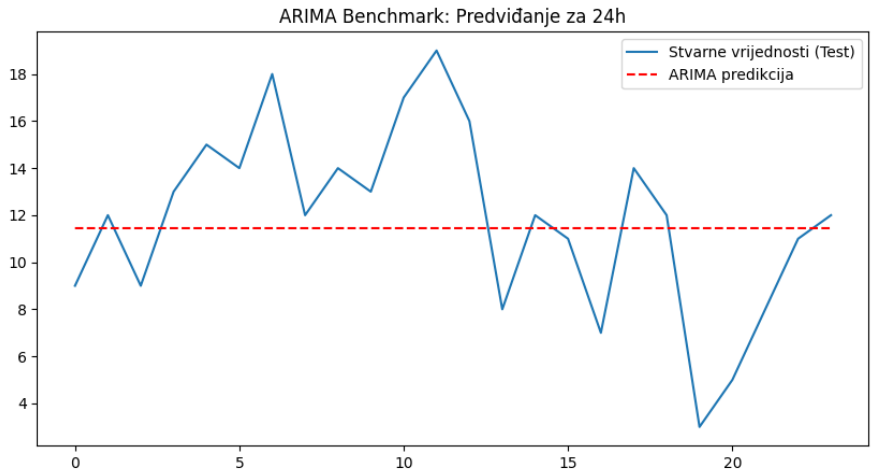
- **p = 1 (AutoRegressive):** Model uzima u obzir vrijednost iz prethodnog sata.
- **d = 1 (Integrated):** Podaci se diferenciraju jednom kako bi se postigla stacionarnost (stabilizacija srednje vrijednosti).
- **q = 1 (Moving Average):** Model koristi grešku iz prethodnog koraka kako bi ispravio trenutnu predikciju.

Posebno važno za performanse na prenosnim računarima je korišćenje metode `innovations_mle`, koja omogućava brže pronalaženje parametara modela bez gubitka značajne preciznosti.

3.2 Rezultati i evaluacija

Evaluacija modela izvršena je na prvih 24 sata testnog skupa podataka. Dobijeni rezultati su:

- **RMSE (Root Mean Squared Error):** 3.87
- **MAE (Mean Absolute Error):** 3.05



Slika 3: Poređenje stvarnih vrijednosti i ARIMA predikcije za period od 24h

Kao što se vidi na Slici 3, ARIMA model teži ka "usrednjavanju"predikcije. Iako su greške RMSE i MAE relativno niske, model ne uspijeva u potpunosti da isprati nagle promjene (šiljke) u zagađenju, što je očekivano za linearne statističke modele.

4 Facebook Prophet Model

Za razliku od klasičnih statističkih modela, **Prophet** tretira predviđanje vremenskih serija kao problem prilagođavanja krive (*curve fitting*), uzimajući u obzir praznike i ciklične komponente na više nivoa.

```python
def run_prophet_model(df):
# Prophet zahtijeva kolone ds (datum) i y (vrijednost)
df_prophet = df.reset_index()[['date', 'PM2.5']]
df_prophet.columns = ['ds', 'y']
df_prophet['ds'] = df_prophet['ds'].dt.tz_localize(None)

# Definisanje modela sa visestrukom sezonalnoscu
model = Prophet(
daily_seasonality=True,
weekly_seasonality=True,
yearly_seasonality=True
)
model.fit(train)

# Generisanje buduceg vremenskog okvira za 24h
future = model.make_future_dataframe(periods=24, freq='H')
forecast = model.predict(future)
```

Listing 6: Implementacija Prophet modela

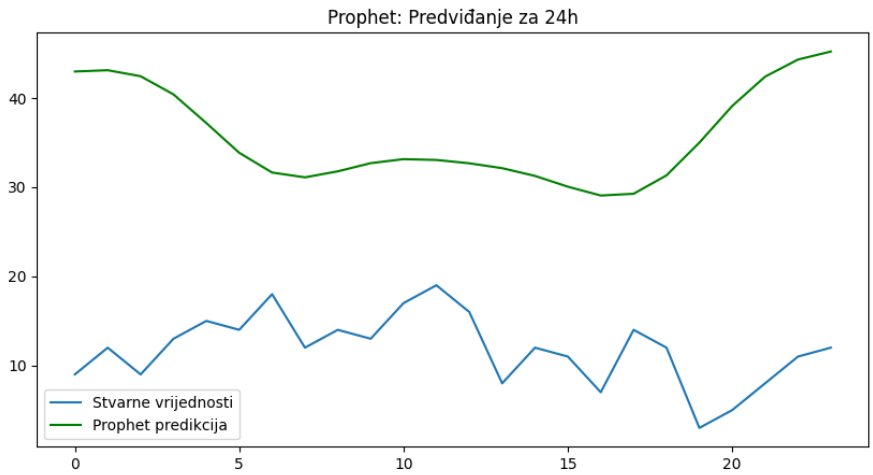
4.1 Rezultati i analiza Prophet modela

Evaluacija modela na testnom skupu dala je sljedeće rezultate:

- **RMSE:** 24.88
- **MAE:** 23.80

4.1.1 Interpretacija lošijih metričkih rezultata

Zanimljivo je uočiti da su RMSE i MAE kod Prophet modela značajno viši nego kod ARIME. Razlozi za ovo leže u samoj prirodi modela:



Slika 4: Predviđanje koncentracije PM2.5 pomoću Prophet modela za period od 24h

- **Preveliko oslanjanje na sezonalnost:** Prophet pokušava da nametne "idealanđnevni i sedmični profil. Ako se u testnom periodu desio nagli skok zagađenja koji nije u skladu sa istorijskim prosjekom za taj sat, Prophet će ga potpuno ignorisati, dok će se ARIMA (koja gleda zadnji sat) brže prilagoditi.

- **Aditivni model:** Prophet najbolje radi na serijama koje imaju jasne trendove rasta kroz godine, dok je zagađenje vazduha veoma haotično i zavisi od trenutnih meteoroloških promjena koje ovaj model ne vidi direktno.

4.1.2 Linijska analiza Prophet implementacije

U nastavku je pruženo detaljno objašnjenje ključnih operacija unutar funkcije `run_prophet_model`:

- `reset_index()`: Izvlači datum iz indeksa tabele i pretvara ga u regularnu kolonu, što je preduslov za rad sa Prophet bibliotekom.

- `columns = ['ds', 'y']`: Vrši esencijalno preimenovanje kolona. Prophet zahtijeva kolonu `ds` (*datestamp*) za vremensku osu i `y` za ciljnu vrijednost (PM2.5).

- `tz_localize(None)`: Uklanja informacije o vremenskoj zoni kako bi se izbjegle nekompatibilnosti tokom proračuna trendova.

- `daily_seasonality=True`: Aktivira ugrađeni Fourierov red koji omogućava modelu da prepozna ciklične promjene unutar 24 časa (npr. jutarnji špic zagađenja).
- `make_future_dataframe`: Generiše novi skup podataka koji sadrži samo datume za budući period (24 sata unaprijed), čime se priprema teren za ekstrapolaciju.
- `predict(future)`: Glavna funkcija koja koristi naučene težine trenda i sezonalnosti kako bi popunila kolonu `yhat` (predviđena vrijednost).

5 LSTM Neuronska Mreža (Deep Learning)

U ovom radu primijenjena je **LSTM (Long Short-Term Memory)** arhitektura, koja pripada porodici rekurentnih neuronskih mreža (RNN). Za razliku od prethodnih modela, LSTM je sposoban da zadrži informacije kroz duže vremenske periode, što je ključno za modeliranje zagađenja vazduha gdje prethodni atmosferski uslovi direktno utiču na trenutno stanje.

```
def run_lstm_model(df):
# 1. Priprema i skaliranje podataka
scaler = MinMaxScaler(feature_range=(0,
    1))
data_scaled = scaler.fit_transform(df[['
    PM2.5']])

# 2. Definisanje 'prozora' (look-back
    perioda)
look_back = 24 # Gledamo 24 sata unazad
    za predikciju narednog sata
test_size = 4320

def create_dataset(dataset, look_back=1):
X, Y = [], []
for i in range(len(dataset)-look_back-1):
X.append(dataset[i:(i+look_back), 0])
Y.append(dataset[i + look_back, 0])
return np.array(X), np.array(Y)

# 3. Podjela na trening i test set
train_data = data_scaled[:-test_size]
test_data = data_scaled[-test_size-
    look_back:]
```

```
X_train, y_train = create_dataset(train_data, look_back)
X_test, y_test = create_dataset(test_data, look_back)

# 4. Reshape u format [uzorci, vremenski koraci, osobine]
X_train = np.reshape(X_train, (X_train.shape[0], X_train.shape[1], 1))
X_test = np.reshape(X_test, (X_test.shape[0], X_test.shape[1], 1))

# 5. Izgradnja arhitekture mreze
model = Sequential([
LSTM(50, activation='relu', input_shape=(look_back, 1)),
Dense(1)
])
model.compile(optimizer='adam', loss='mse')

# 6. Proces treniranja (Fitting)
model.fit(X_train, y_train, epochs=5, batch_size=32, verbose=0)

# 7. Predvidjanje i inverzno skaliranje
predictions = model.predict(X_test[:24], verbose=0)
predictions = scaler.inverse_transform(predictions)
```

Listing 7: Kompletna implementacija LSTM modela

5.1 Analiza LSTM arhitekture i procesa treniranja

Implementacija LSTM modela zahtijevala je kompleksniju transformaciju podataka u odnosu na ARIMA i Prophet modele:

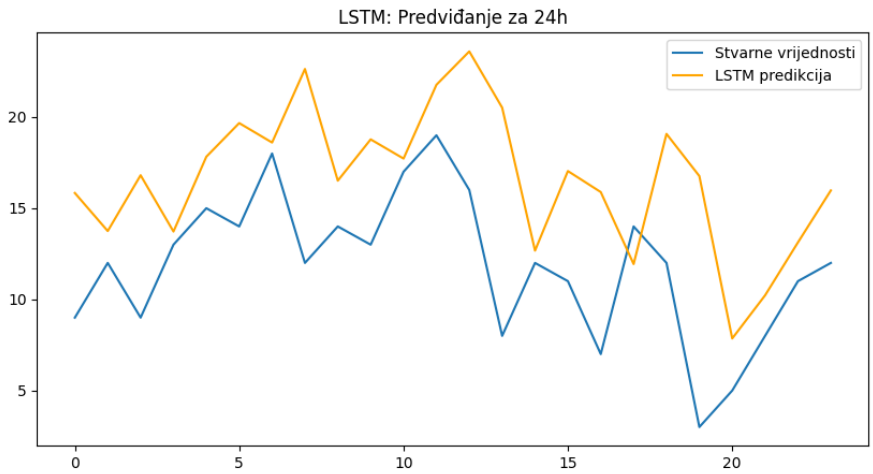
- **Min-Max skaliranje:** Budući da neuronske mreže koriste gradijentne metode optimizacije, podaci su svedeni na opseg $[0, 1]$. Ovo sprječava dominaciju velikih vrijednosti nad malim i ubrzava konvergenciju modela.

- **Klizni prozor (*Sliding Window*):** Funkcija `create_dataset` transformiše seriju u nadgledani problem učenja (*supervised learning*). Svaki ulazni podatak X je zapravo vektor od 24 uzastopna mjerenja, dok je Y ciljna vrijednost u trenutku $t + 1$.

- **Struktura modela:** Odabrano je 50 LSTM jedinica koje formiraju "pamćenje"mreže. Izlazni sloj koristi `Dense` neuron bez aktivacione funkcije, što je standard za probleme regresije (predviđanje kontinuiranih vrijednosti).

5.2 Rezultati i vizuelizacija

Evaluacija modela izvršena je na testnom skupu, pri čemu je dobijena vrijednost korijena srednje kvadratne greške **RMSE: 6.19**.



Slika 5: Prikaz predikcije LSTM modela u odnosu na stvarne vrijednosti PM2.5